Waypoint-2: A Separable Causal Video World Model For Real-Time Streaming On Consumer Devices

Rajit Rajpal* **Shahbuland Matiana*** **Liew Wei Pyn*** **Anmol Agarwal***
Ryan Craig Andrew Lapp Jerry Xiong Mithun Hunsur Scottie Fox Chris Foster
Elizabeth DaCruz Carson Poole Irene Park David Rossi Spencer Frazier Louis Castricato
Overworld
*Equal contribution

Project Page

Abstract

We present **Waypoint-2**: a real-time interactive video world model trained on gameplay and egocentric real-world footage that generates 720p video on consumer GPUs from text, keyboard and mouse input. Waypoint-2's architecture is based on Separable Causal Diffusion (SCD) which splits a causal video diffusion transformer (DiT) model into an encoder which summarizes context frames once per-frame and feeds it to the lightweight decoder that attends only within the current frame and runs at every denoising step. This split moves most of the model's capacity out of the per denoising step cost, so a larger model fits within a fixed frame-rate budget. It also enables a training curriculum that starts with text-to-image (T2I) pretraining of the decoder and moves on to video, first bidirectionally and then causally. On an **RTX 5090**, Waypoint-2 generates 720p video at **75 FPS** including decoding. Measured with the same benchmark, it is faster than Waypoint-1.5 on every GPU we tested despite its larger model and higher resolution, and adheres to keyboard and mouse controls more closely.



Figure 1: **Waypoint-2 teaser.** Two example autoregressive rollouts generated in real time on a single consumer GPU (top and bottom rows). The 1.49B parameter model generates high-resolution video conditioned on text, keyboard and mouse input.

Preprint.

1 Introduction

A video world model generates the next frames of a scene from what has been seen so far and what the user does. When the user is a person at a keyboard and mouse, the model has to respond within the time of a single frame, every frame, for as long as they keep playing. Learned models can now generate explorable worlds in this way [16, 34, 23, 31], but much of this progress assumes datacenter hardware (Table 1). Waypoint-1.5 [30] showed that a 1.28B-parameter model can run in real time on consumer GPUs. This report aims to answer the questions: "how far that can be pushed? how much model fits inside a real-time budget on the hardware people already own?"

A 60 FPS clip generated from latent frames with $4\times$ temporal compression must denoise and decode a new latent frame every 66.7 ms. A causal diffusion model spends that time on several denoising passes per frame where every layer attends to the cached context frames. As a result, the cost of a generated frame grows with the product of depth and step count. Few-step distillation [42, 20] reduces the number of denoising steps but not the cost of each one.

Bai et al. [4] showed that much of this per-step computation is redundant, because standard causal video diffusion models entangle temporal reasoning with iterative denoising: their early layers compute nearly identical features at every denoising step, and their deeper layers rarely attend across frames. Their Separable Causal Diffusion (SCD), on which Waypoint-2 is built, decouples the two. A deep encoder with frame-causal attention reads the past frames, the player's input and the text prompt, and summarizes them into a context embedding for the next frame. Since the encoder runs once per frame and caches its keys and values, no earlier frame is ever encoded twice. A shallow decoder then generates the next frame from this embedding, attending only within that frame, and it is the only component that runs at every denoising step. Capacity placed in the encoder is thus paid for once per frame rather than once per step, which lets a larger model fit within the same budget (Section 4). With this design, Waypoint-2 generates 720p video at 75 FPS on an RTX 5090, including decoding.

This factorization gives the encoder an unusual role. It never sees the frame it helps to generate, yet it is the only path by which the past reaches the decoder. Its output must therefore capture everything about the next frame that can be predicted from the past and the player's input, leaving the generative decoder to supply the rest. In this respect, the encoder is close in spirit to predictive self-supervised learning (Section 8). The factorization also makes a training curriculum possible. Because the decoder only ever sees one frame, it is effectively a conditional image generator, and when no previous frame is available, a learned beginning-of-sequence (BOS) token takes the place of the context. The decoder can therefore be pretrained on its own on text-to-image (T2I) data, which gives it a strong initialization for video generation. The encoder is then added, and the model is trained on video, first with bidirectional attention across frames and then causally, before being distilled into a few-step model (Section 7).

Waypoint-2 is conditioned on a text prompt, raw keyboard and mouse input read once per displayed frame. Many gaming world models instead express actions as camera motion or pose [23, 46, 40, 31, 2], which requires pose labels and cannot represent actions that do not move the camera. Only a few models take raw input directly [30, 37, 26, 31, 33]. Waypoint-2 is also trained on real-world egocentric video alongside gameplay, with the aim of improving generalization.

Table 1: Comparison with other real-time world models. Numbers are as reported by each work, on different hardware and settings. Video FPS counts output frames. For the Waypoint models, measured in BF16 with the same benchmark excluding decoding.

Model	Resolution	Parameters	Action input	Hardware	Video FPS
HY-World 1.5 [34, 33]	720p	8.3B	keys, camera pose	8 H800	24
Hunyuan-GameCraft [23]	720p	13B	camera motion	not stated	6.6
LingBot-World-Fast [31]	480p	14B	keys, camera pose	1 GPU node	16
Genie 3 [16]	720p	not disclosed	navigation, text	8 H100	24
Matrix-Game 3.0 [37]	720p	5B	keyboard, mouse	8 GPUs + 1 for decoding	40
Matrix-Game 3.5 [28]	1280×704	5B	camera pose, text	1 H100	20
Matrix-Game 4.0 [44]	480p	5B	keyboard, mouse (via engine), text	1 RTX 5090	83 (FP8)
Waypoint-1.5 [30]	512×1024	1.28B	keyboard, mouse	1 RTX 5090	66.3
Waypoint-2	**720p**	**1.49B**	**keyboard, mouse, text**	**1 RTX 5090**	**89.8**

Our contributions are as follows:

1. Waypoint-2, to our knowledge the first interactive world model built on SCD, together with an analysis of how the factorization raises model capacity under a real-time budget.
2. A training curriculum that progresses from text-to-image pretraining of the decoder to bidirectional, causal and long-context video training.
3. A real-time 720p world model conditioned on text and on raw keyboard and mouse input, with throughput measured on consumer GPUs. It runs at 75 FPS on an RTX 5090 including decoding, faster than Waypoint-1.5 on every GPU we tested.

2 Related Work

Early world models trained a policy inside a learned model of the environment [18]. Diffusion models later turned such learned simulators into high-fidelity models of single games [3, 35, 29], and the same recipe has since been scaled to open-ended worlds [11, 16, 34, 31, 46, 12], to physical AI [27, 22], and to multiplayer and agent-training settings [32, 1]. Many of these models take the user's input as camera motion or pose [23, 46, 40, 31, 2]. Matrix-Game [37], Yume1.5 [26], LingBot-World (Act) [31], WorldPlay [33] and Waypoint-1.5 [30] instead condition on raw keyboard and mouse input, and Waypoint-1.5 showed that such a model can run in real time on consumer GPUs. Matrix-Game 4.0 [44] also runs in real time on a consumer GPU (Table 1), but the player's keyboard and mouse input drives a game engine, and its video model translates the engine's low-poly render into finished frames, a lighter video-to-video task that it runs in one denoising step instead of four, with fewer tokens per frame (405 against 512), a shorter context (21 latent frames against 68) and FP8 instead of BF16.

Video diffusion models first generated whole clips at once, with every frame attending to every other [7, 19, 36]. Interactive use needs one frame at a time. Self-Forcing [20] and other works [10, 41] distill many-step bidirectional models into few-step causal models using distribution matching distillation (DMD) [42, 43], and condition on their own rollouts during training to reduce autoregressive drift. These methods cut the number of denoising steps, but every step still runs the whole network over the frame and its context. Bai et al. [4] found much of this work to be redundant: early layers compute nearly the same features at every step, and deep layers barely attend across frames. Their Separable Causal Diffusion (SCD) therefore runs a causal encoder once per frame and a lightweight decoder, which attends only within the frame, at every step. SCD was demonstrated on low-resolution video trained from scratch and by converting a pretrained text-to-video model. Waypoint-2 is, to our knowledge, the first interactive world model built on it.

3 Problem Formulation

Let $x_1, \dots, x_T$ denote the latent frames obtained by encoding a video with a video autoencoder (Section 6), where each latent frame covers $\tau = 4$ consecutive video frames. Since the player's input is recorded at every video frame, each latent frame x_t is paired with τ input readings $a_t = (a_t^{(1)}, \dots, a_t^{(\tau)})$, each of which specifies the keys held, the mouse displacement and the scroll direction, if any. A text prompt c_t describes the scene and may change over the course of a rollout. A world model generates latent frames autoregressively, conditioning each frame on all preceding frames and on the inputs and prompts up to the current step:

$$p_\theta(x_{1:T}) = \prod_{t=1}^{T} p_\theta(x_t \mid x_{<t},\, a_{\le t},\, c_{\le t})\,. \tag{1}$$

A rollout may be initialized from one or more seed frames or from the prompt alone.

We model each conditional distribution in Equation (1) with flow matching. Given a clean latent frame x_t and a noise level $\sigma \in [0, 1]$, the noisy frame is obtained by linear interpolation with Gaussian noise,

$$x_t^\sigma = (1 - \sigma)\, x_t + \sigma\, \varepsilon, \qquad \varepsilon \sim \mathcal{N}(0, I), \tag{2}$$

and a network v_θ is trained to predict the velocity $\varepsilon - x_t$ from the noisy frame, the noise level and the conditioning by minimizing

$$\mathcal{L} = \mathbb{E}_{t,\sigma,\varepsilon}\left[\left\| v_\theta(x_t^\sigma, \sigma \mid x_{<t}, a_{\le t}, c_{\le t}) - (\varepsilon - x_t) \right\|^2\right]. \tag{3}$$

During training, σ is sampled uniformly and then shifted toward higher noise levels via $\sigma \leftarrow 3\sigma/(1+2\sigma)$, following common practice for high-resolution generation [13].

The Real-Time Budget. A world model generates f_ℓ latent frames per second and shows $f_p = \tau f_\ell$ video frames per second. Waypoint-2 reads one input per video frame, so its control rate is also f_p, whereas Waypoint-1.5 read input once per latent frame [30]. Showing 60 frames per second requires $f_\ell \geq 15$, which leaves 66.7 ms per latent frame for all N denoising passes and decoding. Our goal is to fit as much model capacity as possible within this budget (Section 4).

4 Separable Causal Diffusion

SCD [4] factorizes the velocity network v_θ, trained with the objective in Equation (3), into an encoder E and a decoder D that serve distinct purposes (Figure 2):

$$h_t = E(x_{<t},\, a_{\leq t},\, c_{\leq t})\,, \qquad \hat{v}_t = D(x_t^\sigma,\, \sigma,\, h_t)\,. \tag{4}$$

The encoder, which runs once per latent frame, maps the history to a context embedding h_t for the next frame and is the only component that propagates information across frames. The decoder then denoises frame t conditioned on h_t, attending only within that frame, and it is the only component evaluated at every denoising step. Our use of SCD departs from the original formulation in how the encoder is trained: whereas Bai et al. [4] train it causally, we first pretrain it with bidirectional attention across frames and make it causal only afterwards (Section 7).

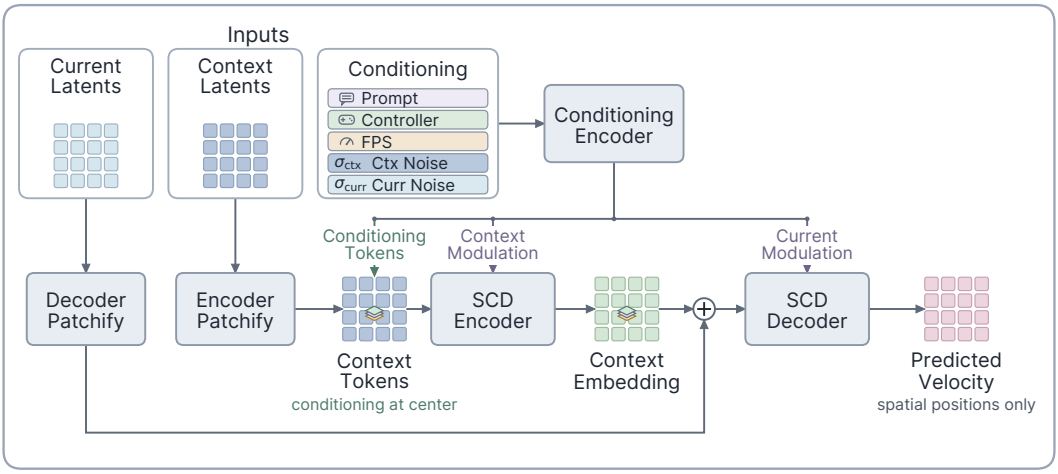


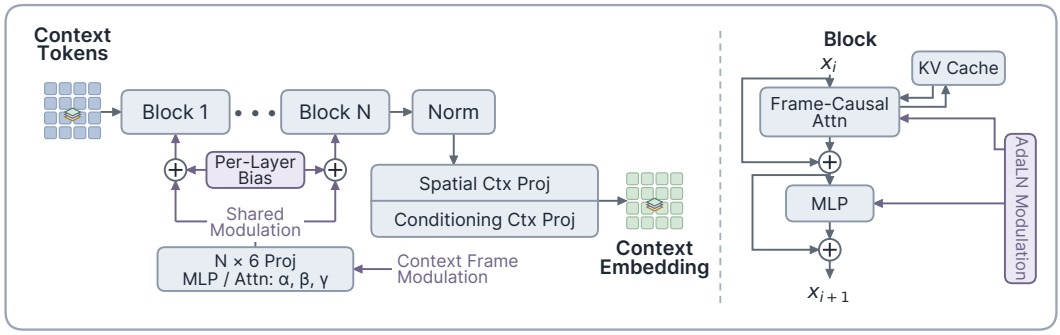
Figure 2: **Waypoint-2 architecture.** The conditioning encoder turns the prompt, controls, frame rate and noise levels into conditioning tokens and modulation vectors. The SCD encoder processes past latents and per-frame conditioning tokens with frame-causal attention and a KV cache, once per frame. Its context embedding is added to the embedded noisy latent of the current frame, and the SCD decoder, which attends only within that frame, predicts the velocity at every denoising step.

Context Encoder

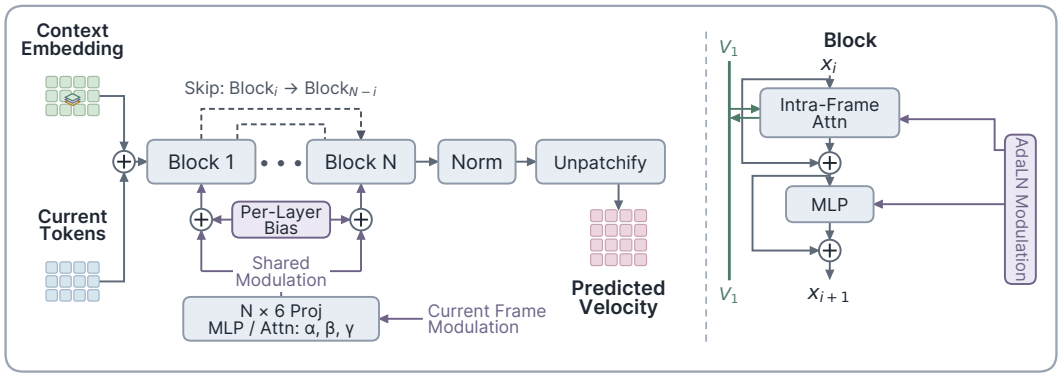
The encoder turns the history into a context embedding for the next frame (Figure 3a). Its input is shifted by one frame: the slot that produces the context for frame t holds frame $t-1$, and the first slot holds only the BOS token (described below). Each slot also carries the conditioning tokens of the frame it predicts. Attention is always bidirectional within a frame, while its pattern across frames changes over the training curriculum (Section 7). In Stage II, where the encoder is first trained on video, attention across frames is bidirectional and the encoder sees a noisy version of every frame in the clip. From Stage III onward it is causal, so the encoder sees only clean past frames, and in Stage IV it is further restricted to a window of recent frames plus a few frames pinned from the start of the rollout, as at inference. In the causal model, the output for frame t depends on $x_{<t}$ and $a_{\leq t}$ but never on x_t, and keys and values are cached so that each frame is encoded exactly once. The final states are normalized and passed to the decoder through two linear projections, one for the spatial tokens and one for the conditioning tokens.

Intra-Frame Decoder

The decoder turns the context embedding into a frame (Figure 3b). At each denoising step it adds an embedding of the noisy latent to the context embedding and processes the result together with the frame's conditioning tokens. Attention never leaves the frame, so the cost of a decoder step does not depend on the length of the rollout. Long skip connections join early and late decoder layers, as in a U-Net, and values from the first layer are mixed into later ones (value residual learning [45]). The decoder is modulated by the noise level and outputs the velocity of Equation (3).



(a) SCD encoder.



(b) SCD decoder.

Figure 3: **SCD encoder and decoder.** (a) The encoder processes the context tokens with frame-causal attention, caching keys and values, and projects its normalized output into the context embedding. (b) The decoder adds the context embedding to the embedded noisy tokens of the current frame and predicts the velocity, with intra-frame attention, skip connections, and a value residual.

Conditioning

Waypoint-2 is conditioned on the signals below (Figure 4). Text and controls enter as tokens attached to each frame. The remaining signals, together with summaries of the text and controls, modulate every layer through AdaLN.

- **Text.** Qwen 3.5 encodes the prompt, and a small transformer with learned register tokens condenses its hidden states into a fixed set of text tokens and a global vector. The prompt can change during a rollout.
- **Controls.** One control token per video frame, built from the held keys, the mouse displacement and the scroll direction. Clips without recorded input use a learned null control.
- **Frame rate.** An embedding of the source frame rate, so that clips recorded at different rates can be mixed.
- **Noise level.** The noise level of the frame being denoised modulates the decoder, and that of the context frames modulates the encoder.

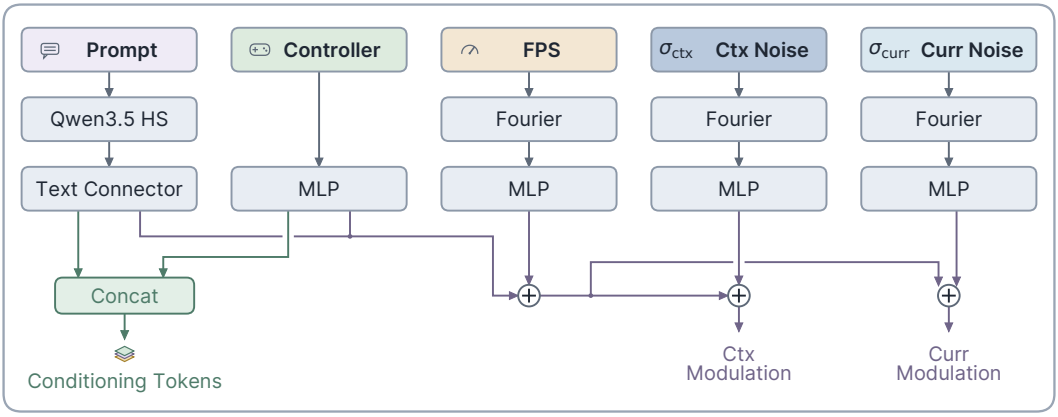


Figure 4: **Conditioning encoder.** The text and control embeddings form the conditioning tokens of each frame. Their summaries are added to embeddings of the frame rate and the noise level.

The BOS Token: Images as One-Frame Videos

The first frame of a rollout has no preceding frame for the encoder to read. This slot is therefore filled with a learned BOS embedding, which is also added to every other encoder input. This design has two consequences: 1) generating the first frame amounts to generating an image from the prompt alone, so T2I generation is the one-frame case of the world model and shares all of its parameters, which makes decoder-only pretraining possible (Section 7), and 2) a rollout can start from text alone, without a seed image.

Cost Analysis: Where to Spend Capacity?

At inference, each latent frame requires one encoder pass over the newest frame, N decoder passes and one autoencoder decode. The encoder pass also appends the frame's keys and values to the cache. Excluding the autoencoder, the cost of a latent frame is therefore $C_{\text{SCD}} = C_E + N\,C_D$, where C_E and C_D denote the costs of one encoder pass and one decoder pass. A single-stream model such as Waypoint-1.5 instead runs every layer with full-context attention at every denoising step, so its entire cost is incurred N times per frame.

Table 2 quantifies this difference for the configuration in Table 3 with a full cache. An encoder pass costs 4.08 TFLOPs, 80% of which is attention over the cached context, while a decoder step costs only 0.49 TFLOPs. With the four denoising steps used at inference, a latent frame therefore costs 6.0 TFLOPs. Running the same 46 layers as a single stream would cost 25.2 TFLOPs, or 4.2 times as much. Equivalently, a single-stream model with the same layer dimensions and the same per-frame cost as Waypoint-2 could have only about 11 layers instead of 46.

Table 2: Forward cost per latent frame in TFLOPs, at steady state with a full context of 64 frames plus 4 pinned frames. We count the matrix multiplications in the transformer layers. The single-stream model has the same 46 layers but runs all of them at every step, each attending to the full context.

	$N=1$	$N=2$	$N=4$	$N=8$	$N=16$	$N=48$
SCD (encoder once, decoder N times)	4.57	5.06	6.04	8.00	11.92	27.58
Single stream, same layers	6.30	12.60	25.20	50.39	100.79	302.36
Ratio	1.4×	2.5×	4.2×	6.3×	8.5×	11.0×
Single-stream depth at equal cost	34	19	11	7	5	4

Two design consequences follow. Depth belongs in the encoder, and the decoder should be as shallow as quality allows: in Waypoint-2 the encoder holds 52% of the parameters, while only the decoder's 30% run at every step. The advantage also grows with the number of steps and with the context length, because only the encoder attends to the context. Conversely, with fewer steps the advantage shrinks, since the encoder term remains. The same asymmetry helps during training, where one encoder pass can serve several decoder noise draws (Section 7). The number of tokens per frame,

set by the autoencoder (Section 6), scales both terms. Measured on real GPUs (Figure 7), the split follows the FLOP count of Table 2: the encoder pass takes 63 to 66% of the time of a latent frame, close to its 68% share of the FLOPs, the four decoder steps take about a quarter, and autoencoder decoding the remaining tenth.

Table 3: Waypoint-2 model configuration.

Component	**Value**
Trainable parameters	1.49B, excluding the frozen text encoder and the autoencoder
Encoder	30 layers, width 1536, 24 query and 12 key-value heads, 0.78B parameters
Decoder	16 layers, width 1536, 12 heads, long skip connections, 0.45B parameters
MLP	expansion $4\times$, SiLU
Cross-frame attention	frame-causal, window of 64 latent frames plus 4 pinned frames
Text encoder	Qwen3.5-4B (frozen), hidden states from 9 layers
Text connector	4 layers, 64 thinking registers, 28 output registers
Tokens per latent frame	512 (480 spatial, 28 text, 4 control)
Output resolution	720×1280
Latent frame	$15 \times 32 \times 128$, covering 4 perceptual frames
Control rate	one control vector per perceptual frame
Positional encoding	2D GGRoPE (space) and 1D RoPE (time)
Noise conditioning	AdaLN with a shared projection and per-layer biases

5 Data

Waypoint-2 is trained on Owl-Control, an internal dataset of gameplay recorded together with keyboard and mouse input, whose collection and preprocessing are described in Waypoint-1.5 [30]. We also mix in real-world egocentric footage.

6 Autoencoder

Waypoint-2 uses a custom causal video autoencoder with a highly compressed latent space, chosen for efficiency and diffusability. It encodes each chunk of four consecutive 720×1280 frames into a 15×32 grid of latent tokens with 128 channels each, an overall compression ratio of $180\times$. Each chunk is encoded *independently*. In our experience, the chunk-causal encoders of existing latent video diffusion models make little use of temporal redundancy across chunks, so removing temporal processing from the encoder saves compute and simplifies its architecture. Moreover, a latent space that genuinely compresses information across time may have reduced diffusability: learned temporal compression can increase the effective dimensionality of the latent and force the diffusion model to attend to a history of latent frames merely to reconstruct the current one.

Causal decoding, in contrast, can improve reconstruction quality even when the encoder does not use information across time. Past reconstructions may carry information that is lost in the lossy latents of later chunks, and they allow decoded frames to transition smoothly across chunk boundaries. Video autoencoders commonly condition on previous *latents* through temporally parallelizable but relatively inexpressive layers such as causal convolutions. Parallelism across time is unimportant for streaming interactive inference, however, so the decoder instead conditions on the last frame of the previous reconstruction, feeding its own last output back as input during both training and inference.

The decoder is trained with a combination of a smoothed L_1 pixel loss and a perceptual feature-matching loss. Both terms use the p-norm distance [15]

$$d(\hat{x}, x) = \frac{\|\hat{x} - x\|_2^2}{\mathrm{sg}\{(\|\hat{x} - x\|_2^2 + \epsilon)^{-p}\}},$$

where sg denotes the stop-gradient operator, $\epsilon = 10^{-3}$ and $p = 0.5$. The overall loss is

$$\mathcal{L}_{\text{AE}} = d(\hat{x}, x) + \lambda_{\text{perceptual}}\, d(\phi(\hat{x}), \phi(x)),$$

where x is the original frame, $\hat{x}$ its reconstruction, ϕ a perceptual feature extractor (DINOv2 ViT-B/14), and $\lambda_{\text{perceptual}} = 0.2$ weights the two terms.

The encoder and decoder are loosely based on HDiT [9]. Both consist of a hierarchy of neighborhood attention blocks with RMSNorm and GGRoPE [39], with space-to-channel rearrangements between adjacent encoder levels and channel-to-space rearrangements between adjacent decoder levels. Instead of skip connections, each decoder level begins with a linear interpolation between its hidden state and an embedding of the corresponding patch of the previous frame. Latents are normalized to unit RMS along the channel dimension during both training and inference. During encoder training, the latents are linearly interpolated with a random amount of Gaussian noise. In a separate finetuning phase, the encoder is frozen, latent noising is disabled, and only the autoregressive decoder is trained to maximize reconstruction fidelity.

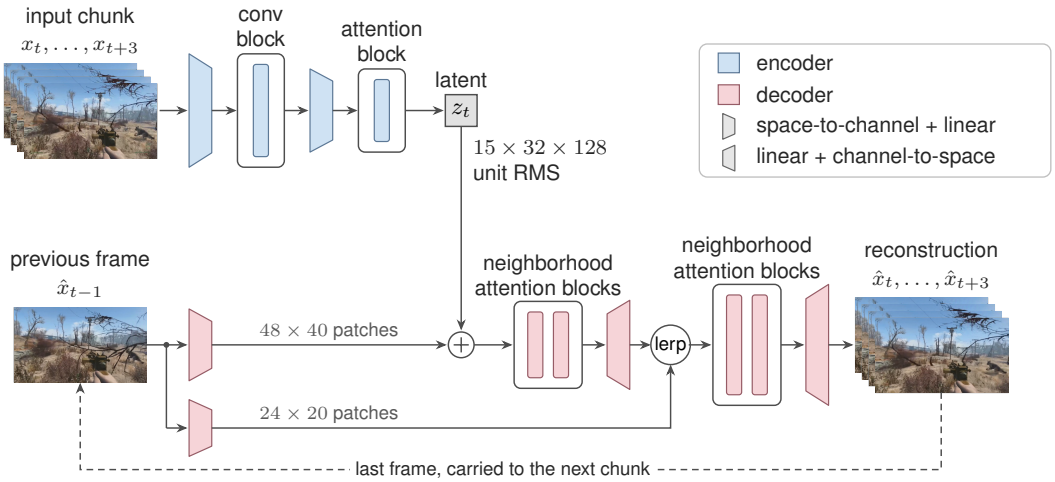


Figure 5: **Autoencoder.** Each chunk of four frames is encoded independently into a $15 \times 32 \times 128$ latent with unit RMS. The decoder reconstructs the chunk from this latent and from the last frame of the previous reconstruction, which enters every decoder level as an embedding of its patches at that level's resolution. Two of the four levels of each network are shown.

Table 4: Reconstruction quality on 250 held-out gameplay windows (121 frames each), against the 720×1280 ground truth, over the pixels that change during each window. TAEHV runs at 512×1024 inside Waypoint-1.5, and its output is resized back to 720p. Samples are in Appendix B.

Autoencoder	Resolution	Compression	PSNR ↑	SSIM ↑	LPIPS ↓
TAEHV (Waypoint-1.5) [6]	512×1024	$96\times$	28.46	0.835	**0.238**
Waypoint-2	720×1280	$180\times$	**29.69**	**0.872**	0.252

7 Training

Waypoint-2 is trained in four stages that move from single images to long causal rollouts (Figure 6). Each stage starts from the weights of the previous one. All stages use the flow matching objective of Equation (3), with NorMuon [24, 21] for matrix-shaped weights and AdamW for the rest. Appendix A lists the hyperparameters of each stage.

Stage I: Text-to-Image Pretraining of the Decoder

The first stage only trains the decoder and the text-conditioning on images. The encoder is left untrained so the decoder sees only the BOS and conditioning tokens as context, effectively making it a T2I DiT (Section 4). This gives a strong initialization for T2V continued pretraining in the next stage. The prompt is dropped for a fraction of samples to enable CFG. We add the Self-Flow representation loss [8], which aligns intermediate decoder features with deeper features of an exponential moving average (EMA) of the model that sees a less noisy version of the input.

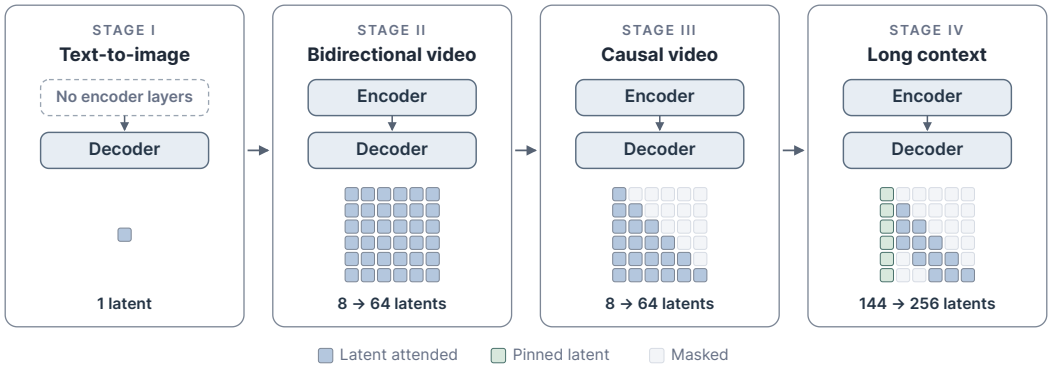


Figure 6: **Training curriculum.** The decoder is first trained alone on images, and the encoder is added in Stage II. Grids show which latents each latent can attend to, with rows for the latent being generated and columns for the latents it sees. Attention across latents moves from bidirectional to causal, and the last stage trains with a rolling window and pinned latents to match inference.

Stage II: Bidirectional Video Pretraining

We then train the encoder and zero-initialize its AdaLN gates and the projection layer. At the beginning of training, no information passes between frames, and each frame is generated from BOS and its own conditioning as in Stage I.

Attention across frames is bidirectional in this stage. All frames of a clip share one noise level and the encoder sees their noisy versions, so the model is a bidirectional video diffusion model split into an encoder and a decoder. Up to two leading frames are kept clean, which teaches the model to continue from given frames. Training moves from short clips to long, captioned clips. Controls are dropped independently of the text, and more often in the long-clip phase, so that guidance can be applied to each condition separately. Self-Flow continues throughout. The causal constraint is imposed only in Stage III, as in prior work that adapts bidirectional video diffusion models into causal ones [43].

Stage III: Causal Conversion

We finetune the bidirectional model into a causal one by applying a teacher-forced frame-causal attention mask to the encoder. Because the encoder input is shifted by one frame, the context for frame t then depends only on $x_{<t}$. Because the decoder works on a single frame and is cheap, each encoder pass is reused for several independent decoder noise draws, which gives the encoder more training signal per pass. As in Stage II, training moves from short to long clips. Self-Flow is no longer used.

Stage IV: Long Context with Pinned Frames

The last pretraining stage trains with the cache layout used at inference. Older frames leave a rolling window, while the BOS slot and the first few frames stay pinned and remain visible to every later frame. Temporal positions of the pinned frames are shifted so that their distance from the current frame stays within context window. Sequences grow to 256 latent frames, about 17 seconds of video.

Distillation

Finally, we distill the causal model into a student that generates each frame in four denoising steps, using a custom scheme built on recent few-step methods. The student is first trained with MeanFlow [14] to take large steps along the flow, with classifier-free guidance for text and controls folded into its predictions, so inference needs a single forward pass per step. It is then refined on its own autoregressive rollouts, as in Self-Forcing [20], using distribution matching distillation [42] against a frozen causal teacher and a trainable critic, in the manner of AnyFlow [17]. As in Solaris [32], the history of each rollout is generated without gradients and then re-encoded with gradients, so that the loss reaches the encoder through its view of the past. We leave out the remaining details of the recipe.

8 Experiments

Real-Time Performance

Table 5 reports throughput across GPUs, with Waypoint-1.5 as a reference, and Figure 7 splits the time of one latent frame between the encoder, the decoder and the autoencoder.

Table 5: Throughput across consumer GPUs, in video frames per second (four per latent frame). Both models are measured with the Waypoint-1.5 benchmark, in BF16, from an empty cache and without decoding. Waypoint-1.5 numbers are from [30]. The last column includes autoencoder decoding and is measured at steady state, with a full context window.

	Generation only		With decoding
Hardware	**Waypoint-1.5**	**Waypoint-2 (720p)**	**Waypoint-2 (720p)**
RTX PRO 6000 Blackwell	87.76	123.42	101.04
RTX 5090	66.28	89.80	74.62
RTX 4090	66.60	71.67	59.08
RTX 3090	21.60	35.12	28.58
RTX 3070	14.45	20.50	16.87
RTX 3060	8.80	14.28	11.72
H200 (datacenter, for reference)	168.86	234.80	151.53

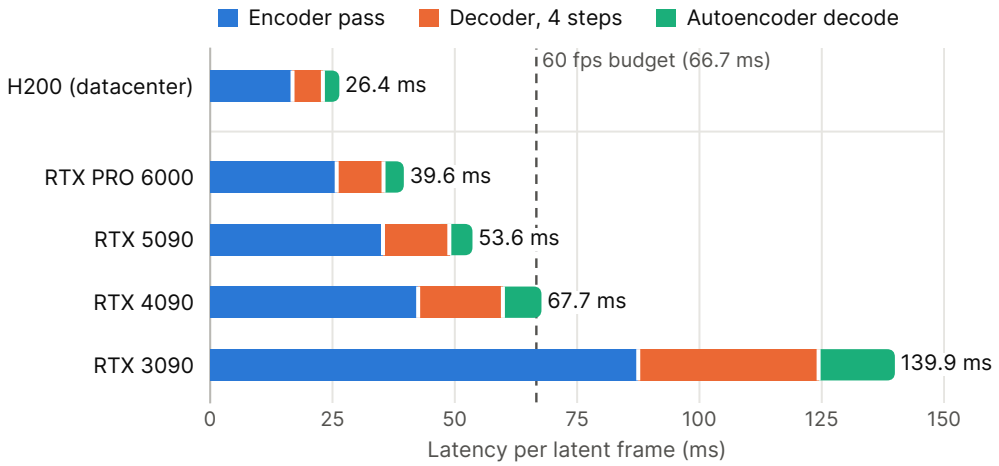


Figure 7: **Where the time goes.** Latency per latent frame on consumer GPUs and, for reference, a datacenter H200, split into the encoder pass, the four decoder steps and autoencoder decoding. The dashed line marks the 66.7 ms budget for 60 frames per second.

Generation Quality

Table 6: Stage I T2I quality on 10,000 held-out prompts. 50 Euler steps, CFG scale 5, no schedule shift.

Model	**FID ↓**	**SigLIP 2 ↑**
Real images, autoencoded	1.49	7.51
Stage I decoder	10.29	8.55

Table 6 and Figure 8 show text-to-image quality after Stage I. The decoder reaches an FID of 10.3, against 1.5 for real images passed through the autoencoder, which is the floor for any generator in this latent space. Its SigLIP 2 score is higher than that of the real images (8.55 against 7.51), so the samples match the prompts more closely than the held-out images the prompts describe. Both are measured with classifier-free guidance (CFG) at a scale of 5, which favors prompt adherence over

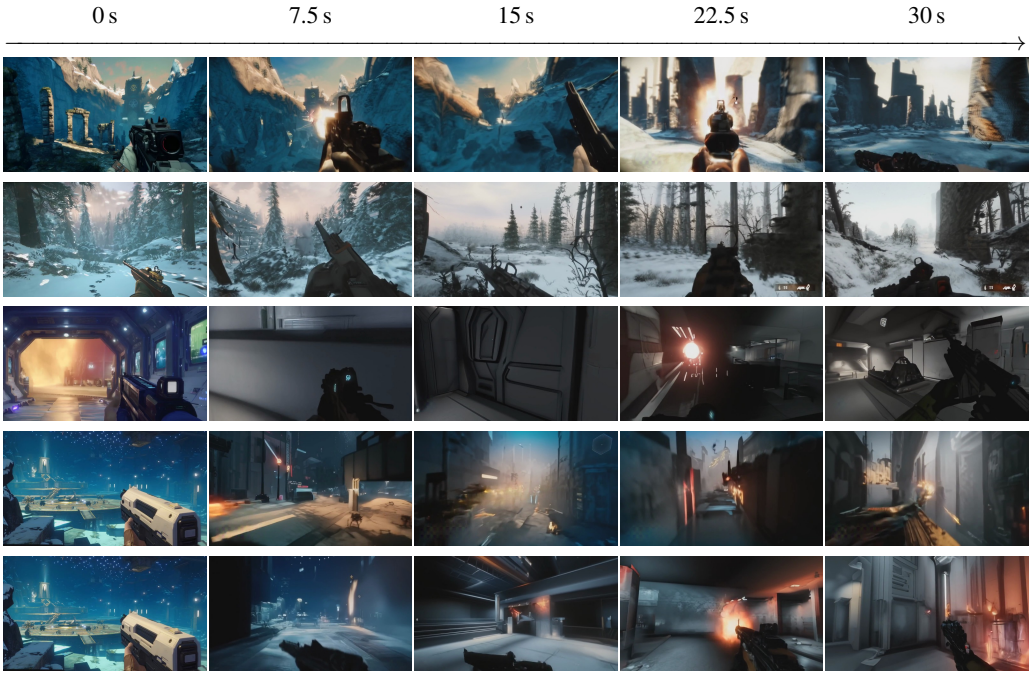


Figure 9: **Waypoint-2 autoregressive rollouts.** Each row is a 30-second rollout.

sample diversity and raises FID. Table 7 reports video quality by training stage and conditioning mode, and Figure 9 shows qualitative rollouts.

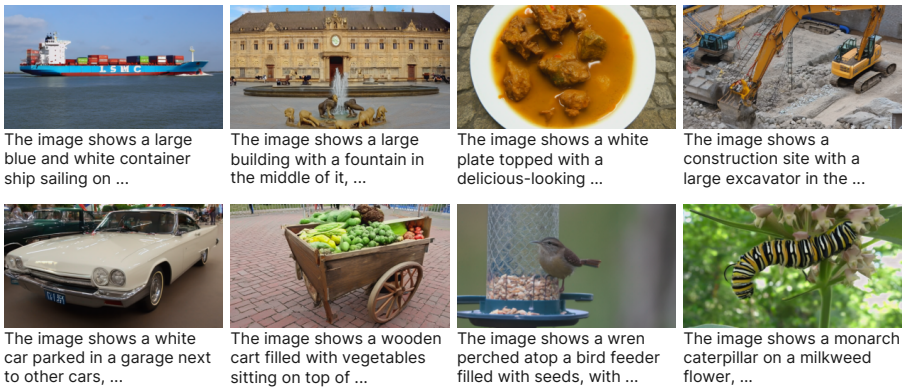


Figure 8: **Stage I samples.** T2I samples from decoder after Stage I, at 720p with 50 steps and a guidance scale of 5. Prompts are captions of held-out images, shortened here.

Table 7: Video quality on 256 held-out two-second gameplay clips, as KDD / JEDi (lower is better).

Model	**Text and controls**	**Text only**	**Controls only**	**Seedless**
End of Stage II (bidirectional)	0.39 / 0.09	1.12 / 0.55	0.48 / 0.10	1.07 / 0.19
End of Stage III (causal)	1.43 / 0.18	1.18 / 0.44	1.46 / 0.18	1.47 / 0.21
End of Stage IV (long context)	1.67 / 0.22	1.76 / 0.43	1.62 / 0.21	1.66 / 0.26
Distilled model	0.45 / 0.09	0.97 / 0.34	0.53 / 0.09	1.50 / 0.24
Waypoint-1.5 [30], inference engine	n/a	n/a	0.66 / 0.22	n/a
Waypoint-2, inference engine	0.67 / 0.10	–	0.65 / 0.09	–

Control Fidelity

We measure how closely generated video follows the input with two models that only see the video. An inverse dynamics model (IDM) [5], trained on Owl-Control, predicts for every frame which keys are held and how the mouse moves, and Depth Anything 3 [25] estimates the camera rotation. We take 104 held-out four-second windows in which the player moves and turns, seed each model with the real first frame, and drive it with the recorded input, together with the caption for Waypoint-2, which is the only model with text conditioning. Waypoint-2 follows the input about as closely as the game itself. The IDM recovers its keys and mouse direction at least as well as on real footage, and its camera turns with the mouse (rotation agreement 0.87, against 0.88 for real video). Waypoint-1.5 follows the mouse less closely, with more than twice the direction error and lower rotation agreement.

Table 8: Control fidelity on 104 held-out four-second windows. Key F1 compares IDM's WASD predictions with input keys. Mouse direction error is median angle between predicted and input mouse motion. Rotation agreement is median correlation between yaw from DA3 and mouse motion.

Model	Key F1 ↑	Mouse direction error ↓	Rotation agreement ↑
Real video (reference)	0.86	2.0°	0.88
Waypoint-1.5 [30]	0.88	4.3°	0.64
Waypoint-2	**0.91**	**1.8°**	**0.87**

The BOS Token

We study how the encoder uses BOS in text-only rollouts of the distilled model (Figure 12), with captions of held-out clips and scripted input (walking forward while turning). Every later frame keeps attending to BOS (Figure 10a). Once the window is full, BOS receives 1.4 times the attention of an average cached frame, and the other three pinned frames about half as much. Most of this attention comes from the first four encoder layers, which give BOS 3.5 times the average share (Figure 10b). A few heads that resemble the attention sinks of language models [38] also contribute. One of them places about half of its attention on BOS. To isolate the role of the pinned frames, we: 1) remove either BOS alone or 2) all pinned frames from attention at the point where a rolling cache would evict them. The three ablations perturb the rollouts similarly. Over a longer horizon, removing BOS has the strongest destabilizing effect.

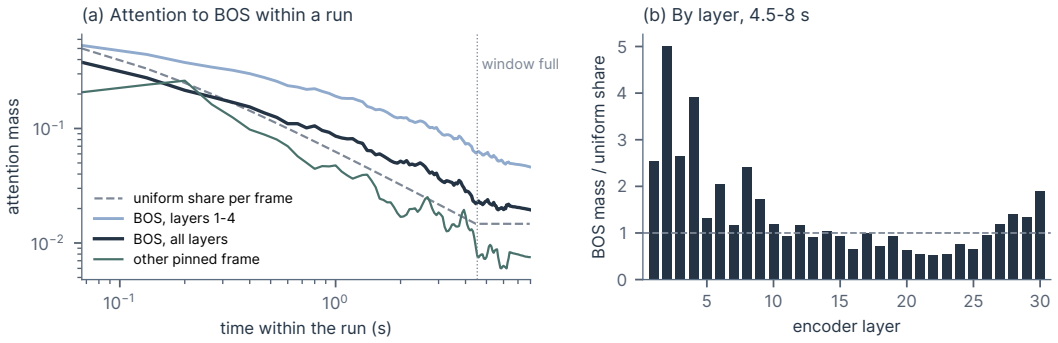


Figure 10: **BOS token.** (a) Attention that encoder places on BOS and on other pinned frames within a run, averaged over eight rollouts, with share each frame would receive under uniform attention. (b) BOS attention by encoder layer once window is full, relative to uniform share.

The SCD Encoder as a Self-Supervised Predictor

The encoder learns only through the generative loss, and its output for frame t is computed before frame t is seen. If the encoder acts as a predictor, this output should encode how the camera moves between frames $t-1$ and t. We test this with linear probes. We average the encoder features at the position of frame t over spatial tokens and regress them onto the camera rotation and translation between frames $t-1$ and t with ridge regression. The camera motion is estimated from the real video

		Encoder					
	Input	**Layer 5**	**Layer 10**	**Layer 15**	**Layer 20**	**Layer 25**	**Output**
Yaw	0.75	0.58	0.71	**0.82**	0.80	0.80	0.70
Pitch	**0.54**	0.30	0.40	0.48	0.50	0.45	0.40
Translation	**0.44**	0.17	0.30	0.39	0.43	0.40	0.36

Table 9: Held-out R^2 of linear probes for camera motion between consecutive latent frames. Input is keyboard and mouse input of the current and a few earlier frames, together with the true camera motion one step earlier.

with Depth Anything 3 [25]. We probe the causal model at the end of Stage III on 240 gameplay clips, split by clip into training, validation and test sets. As a baseline, the same probe is fit to the keyboard and mouse input of frame t and the few frames before it, together with the true camera motion between frames $t-2$ and $t-1$. Features from the middle of the encoder predict yaw better than this baseline (R^2 of 0.82 against 0.75), and they match it for pitch and translation (Table 9). Decodability rises with depth up to the middle of the encoder and falls toward its output.

9 Conclusion

We presented Waypoint-2, a real-time video world model built on SCD. Encoding the context once per frame and denoising with a shallow decoder lets most of the parameters sit where they run only once per frame. The same split allows a curriculum that pretrains the decoder on text-to-image data, and it turns the encoder into a predictive representation learner whose middle layers predict how the camera turns better than the controls and the previous motion do. Waypoint-2 generates 720p video from text, keyboard and mouse input at 75 FPS on an RTX 5090, follows keyboard and mouse input about as closely as real gameplay does, and after distillation into a four-step model comes close to the video quality of the bidirectional Stage II model. Memory beyond the attention window, longer horizons, quantization for smaller GPUs and support for more input devices are left for future work.

References

[1] Anmol Agarwal, Pranay Meshram, Sumer Singh, Saurav Suman, Andrew Lapp, Shahbuland Matiana, Louis Castricato, and Spencer Frazier. Combat: Conditional world models for behavioral agent training. *arXiv preprint arXiv:2603.00825*, 2026.

[2] AlayaWorld Team. Alayaworld: Long-horizon and playable video world generation. *arXiv preprint arXiv:2607.06291*, 2026.

[3] Eloi Alonso, Adam Jelley, Vincent Micheli, Anssi Kanervisto, Amos Storkey, Tim Pearce, and François Fleuret. Diffusion for world modeling: Visual details matter in atari. *Advances in Neural Information Processing Systems*, 37, 2024.

[4] Xingjian Bai, Guande He, Zhengqi Li, Eli Shechtman, Xun Huang, and Zongze Wu. Causality in video diffusers is separable from denoising. In *Conference on Computer Vision and Pattern Recognition 2026*, 2026.

[5] Bowen Baker, Ilge Akkaya, Peter Zhokhov, Joost Huizinga, Jie Tang, Adrien Ecoffet, Brandon Houghton, Raul Sampedro, and Jeff Clune. Video pretraining (VPT): Learning to act by watching unlabeled online videos. In *Advances in Neural Information Processing Systems*, volume 35. Curran Associates, Inc., 2022.

[6] Ollin Boer Bohan. TAEHV: Tiny AutoEncoder for Hunyuan Video. https://github.com/madebyollin/taehv, 2025. Accessed: 2026-06-10.

[7] Tim Brooks, Bill Peebles, Connor Holmes, Will DePue, Yufei Guo, Li Jing, David Schnurr, Joe Taylor, Troy Luhman, Eric Luhman, Clarence Ng, Ricky Wang, and Aditya Ramesh. Video generation models as world simulators. 2024. URL https://openai.com/research/video-generation-models-as-world-simulators.

[8] Hila Chefer, Patrick Esser, Dominik Lorenz, Dustin Podell, Vikash Raja, Vinh Tong, Antonio Torralba, and Robin Rombach. Self-supervised flow matching for scalable multi-modal synthesis. *arXiv preprint arXiv:2603.06507*, 2026.

[9] Katherine Crowson, Stefan Andreas Baumann, Alex Birch, Tanishq Mathew Abraham, Daniel Z Kaplan, and Enrico Shippole. Scalable high-resolution pixel-space image synthesis with hourglass diffusion transformers. *arXiv preprint arXiv:2401.11605*, 2024.

[10] Justin Cui, Jie Wu, Ming Li, Tao Yang, Xiaojie Li, Rui Wang, Andrew Bai, Yuanhao Ban, and Cho-Jui Hsieh. Self-forcing++: Towards minute-scale high-quality video generation. In *The Fourteenth International Conference on Learning Representations*, 2026.

[11] Decart. Oasis: A universe in a transformer, 2024. URL https://oasis-model.github.io/.

[12] DreamX Team. Dreamx-world 1.0: A general-purpose interactive world model. *arXiv preprint arXiv:2606.16993*, 2026.

[13] Patrick Esser, Sumith Kulal, Andreas Blattmann, Rahim Entezari, Jonas Müller, Harry Saini, Yam Levi, Dominik Lorenz, Axel Sauer, Frederic Boesel, et al. Scaling rectified flow transformers for high-resolution image synthesis. In *Forty-first international conference on machine learning*, 2024.

[14] Zhengyang Geng, Mingyang Deng, Xingjian Bai, Zico Kolter, and Kaiming He. Mean flows for one-step generative modeling. *Advances in Neural Information Processing Systems*, 38, 2025.

[15] Zhengyang Geng, Ashwini Pokle, Weijian Luo, Justin Lin, and Zico Kolter. Consistency models made easy. In *International Conference on Learning Representations*, 2025.

[16] Google DeepMind. Genie 3: A new frontier for world models. 2025. URL https://deepmind.google/discover/blog/genie-3-a-new-frontier-for-world-models/.

[17] Yuchao Gu, Guian Fang, Yuxin Jiang, Weijia Mao, Song Han, Han Cai, and Mike Zheng Shou. Anyflow: Any-step video diffusion model with on-policy flow map distillation. *arXiv preprint arXiv:2605.13724*, 2026.

[18] David Ha and Jürgen Schmidhuber. World models. *arXiv preprint arXiv:1803.10122*, 2018.

[19] Yoav HaCohen, Nisan Chiprut, Benny Brazowski, Daniel Shalem, Dudu Moshe, Eitan Richardson, Eran Levin, Guy Shiran, Nir Zabari, Ori Gordon, et al. Ltx-video: Realtime video latent diffusion. *arXiv preprint arXiv:2501.00103*, 2024.

[20] Xun Huang, Zhengqi Li, Guande He, Mingyuan Zhou, and Eli Shechtman. Self forcing: Bridging the train-test gap in autoregressive video diffusion. *Advances in Neural Information Processing Systems*, 38, 2025.

[21] Keller Jordan, Yuchen Jin, Vlado Boza, Jiacheng You, Franz Cesista, Laker Newhouse, and Jeremy Bernstein. Muon: An optimizer for hidden layers in neural networks, 2024. URL https://kellerjordan.github.io/posts/muon/.

[22] Kairos Team. Kairos: A regret-aware native world-action model stack for physical ai. *arXiv preprint arXiv:2606.16533*, 2026.

[23] Jiaqi Li, Junshu Tang, Zhiyong Xu, Longhuang Wu, Yuan Zhou, Shuai Shao, Tianbao Yu, Zhiguo Cao, and Qinglin Lu. Hunyuan-gamecraft: High-dynamic interactive game video generation with hybrid history condition. *arXiv preprint arXiv:2506.17201*, 2025.

[24] Zichong Li, Liming Liu, Chen Liang, Weizhu Chen, and Tuo Zhao. Normuon: Making muon more efficient and scalable. *arXiv preprint arXiv:2510.05491*, 2025.

[25] Haotong Lin, Sili Chen, Junhao Liew, Donny Y Chen, Zhenyu Li, Guang Shi, Jiashi Feng, and Bingyi Kang. Depth anything 3: Recovering the visual space from any views. *arXiv preprint arXiv:2511.10647*, 2025.

[26] Xiaofeng Mao, Zhen Li, Chuanhao Li, Xiaojie Xu, Kaining Ying, and Kaipeng Zhang. Yume1.5: A text-controlled interactive world generation model. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2026.

[27] NVIDIA. Cosmos 3: Omnimodal world models for physical ai. *arXiv preprint arXiv:2606.02800*, 2026.

[28] Runjia Qian, Zile Wang, Jihai Zhang, Kai Zou, Wei Yu, Jiaxing Li, Zexiang Liu, Yaokun Li, Fei Kang, Kaichen Huang, et al. Matrix-game 3.5: Enhancing real-time streaming interactive world models with patch memory. *arXiv preprint arXiv:2608.29910*, 2026.

[29] Rajit Rajpal. Decoupling appearance and geometry in diffusion world models. *arXiv preprint arXiv:2610.20625*, 2026.

[30] Rajit Rajpal, Shahbuland Matiana, Liew Wei Pyn, Anmol Agarwal, Ryan Craig, Andrew Lapp, Mithun Hunsur, Sami BuGhanem, Scottie Fox, Aaron Sanders, Carson Poole, Irene Park, Dave Rossi, Spencer Frazier, and Louis Castricato. Waypoint-1.5: A real-time video world model for consumer hardware. *arXiv preprint arXiv:2609.37107*, 2026.

[31] Robbyant Team. Advancing open-source world models. *arXiv preprint arXiv:2601.20540*, 2026.

[32] Georgy Savva, Oscar Michel, Daohan Lu, Suppakit Waiwitlikhit, Timothy Meehan, Dhairya Mishra, Srivats Poddar, Jack Lu, and Saining Xie. Solaris: Building a multiplayer video world model in minecraft. *arXiv preprint arXiv:2602.22208*, 2026.

[33] Wenqiang Sun, Haiyu Zhang, Haoyuan Wang, Junta Wu, Zehan Wang, Zhenwei Wang, Yunhong Wang, Jun Zhang, Tengfei Wang, and Chunchao Guo. Worldplay: Towards long-term geometric consistency for real-time interactive world modeling. *arXiv preprint arXiv:2512.14614*, 2025.

[34] Team HunyuanWorld. Hy-world 1.5: A systematic framework for interactive world modeling with real-time latency and geometric consistency. Technical report, 2025. URL https://3d-models.hunyuan.tencent.com/world/world1_5/HYWorld_1.5_Tech_Report.pdf.

[35] Dani Valevski, Yaniv Leviathan, Moab Arar, and Shlomi Fruchter. Diffusion models are real-time game engines. In *International Conference on Learning Representations*, 2025.

[36] Team Wan, Ang Wang, Baole Ai, Bin Wen, Chaojie Mao, Chen-Wei Xie, Di Chen, Feiwu Yu, Haiming Zhao, Jianxiao Yang, et al. Wan: Open and advanced large-scale video generative models. *arXiv preprint arXiv:2503.20314*, 2025.

[37] Zile Wang, Zexiang Liu, Jiaxing Li, Kaichen Huang, Baixin Xu, Fei Kang, Mengyin An, Peiyu Wang, Biao Jiang, Yichen Wei, et al. Matrix-game 3.0: Real-time and streaming interactive world model with long-horizon memory. *arXiv preprint arXiv:2604.08995*, 2026.

[38] Guangxuan Xiao, Yuandong Tian, Beidi Chen, Song Han, and Mike Lewis. Efficient streaming language models with attention sinks. In *International Conference on Learning Representations*, 2024.

[39] Jerry Xiong. On n-dimensional rotary positional embeddings, 2025. URL https://jerryxio.ng/posts/nd-rope/.

[40] Jiacong Xu, Hanwen Jiang, Zhixin Shu, Kalyan Sunkavalli, Vishal M Patel, and Yiqun Mei. Wonder: Video world model done better. *arXiv preprint arXiv:2607.26037*, 2026.

[41] Shuai Yang, Wei Huang, Ruihang Chu, Yicheng Xiao, Yuyang Zhao, Xianbang Wang, Muyang Li, Enze Xie, Yingcong Chen, Yao Lu, et al. Longlive: Real-time interactive long video generation. *arXiv preprint arXiv:2509.22622*, 2025.

[42] Tianwei Yin, Michaël Gharbi, Taesung Park, Richard Zhang, Eli Shechtman, Fredo Durand, and William T Freeman. Improved distribution matching distillation for fast image synthesis. *Advances in neural information processing systems*, 37, 2024.

[43] Tianwei Yin, Qiang Zhang, Richard Zhang, William T Freeman, Fredo Durand, Eli Shechtman, and Xun Huang. From slow bidirectional to fast autoregressive video diffusion models. In *2025 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. IEEE, 2025.

[44] Jihai Zhang, Runjia Qian, Kaiqin Kong, Biao Jiang, Puyi Wang, Siru Chen, Tianle Li, Guanjie Chen, Junyi Wu, Chubin Chen, Yuanhang Li, Kaichen Huang, Haobo Zhang, Wei Yu, Guibin Chen, Yu Cheng, and Yahui Zhou. Matrix-game 4.0: Engine-grounded neural rendering for real-time interactive worlds. Technical report, 2026. URL https://matrix-game-v4-0.github.io/page/.

[45] Zhanchao Zhou, Tianyi Wu, Zhiyun Jiang, Fares Obeid, and Zhenzhong Lan. Value residual learning. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 2025.

[46] Haoyi Zhu, Haozhe Liu, Yuyang Zhao, Tian Ye, Junsong Chen, Jincheng Yu, Tong He, Song Han, and Enze Xie. Sana-wm: Efficient minute-scale world modeling with hybrid linear diffusion transformer. *arXiv preprint arXiv:2605.15178*, 2026.

A Training Hyperparameters

All stages use NorMuon (learning rate 10^{-3}, momentum 0.95, weight decay 0.01) for 2D weight matrices and AdamW (learning rate 10^{-4}, $\beta = (0.9, 0.95)$, weight decay 0.01) for the remaining parameters, with noise levels drawn as in Section 3.

Table 10: Training hyperparameters by stage. Clip and window lengths are in latent frames. The global batch is the number of sequences per optimizer step. In Stage I a sequence is a pack of up to 64 images. Steps are optimizer steps. Stage III starts from the Stage II (short) model.

	I	**II (short)**	**II (long)**	**III (short)**	**III (long)**	**IV**
Attention across frames	–	bidirectional	bidirectional	causal	causal	causal
Window	–	8	64	8	64	64, plus 4 pinned
Clip length	1	8	64	8	64	144 to 256
Global batch	64	64	64	16	32	32
Decoder draws K	1	1	1	4	4	1
Self-Flow token ratio	0.2	0.1	0.1	off	off	off
Clean seed frames	–	up to 2	up to 2	–	–	–
Text dropout	0.1	0.1	0.1	0.1	0.1	0.1
Control dropout	–	0.1	0.5	0.1	0.1	0.1
Steps	350k	425k	50k	25k	50k	50k

B Autoencoder Reconstructions

Figure 11 shows held-out frames reconstructed by the Waypoint-2 autoencoder and by TAEHV, the autoencoder of Waypoint-1.5.

C Additional Samples

Figure 12 shows the first frames that the distilled model generates from a prompt alone, with no seed image starting from the BOS token.

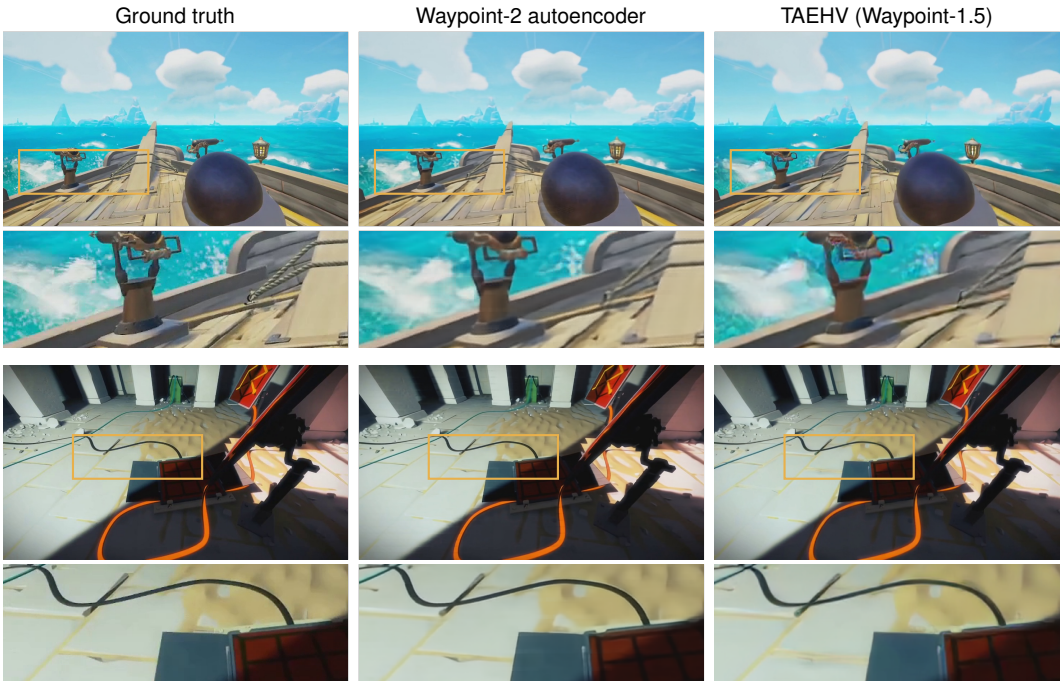


Figure 11: **Autoencoder reconstructions.** Held-out frames (left), reconstructed by the Waypoint-2 autoencoder (middle) and by TAEHV, the autoencoder of Waypoint-1.5 (right). Boxes mark the zoomed crops.

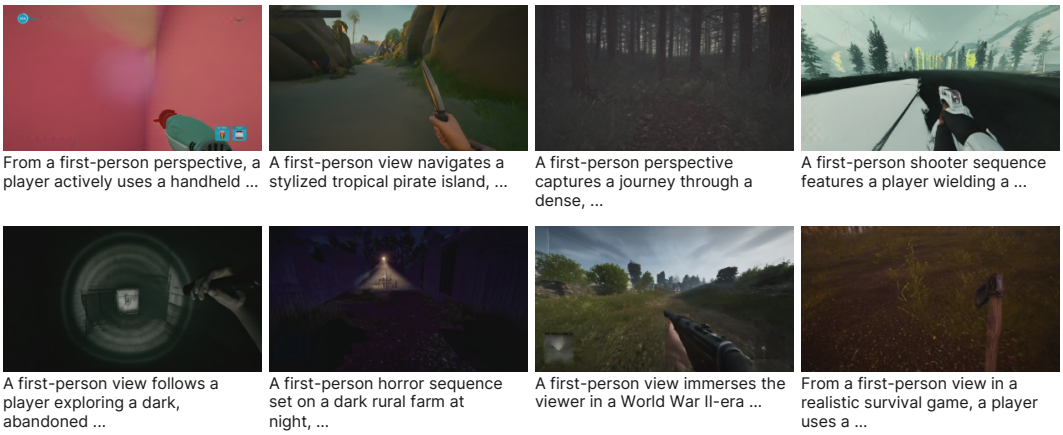


Figure 12: **Text-to-image samples from the distilled model.** First frames generated from a prompt, with no seed image, in four denoising steps at 720p.